

汽车销售管理系统课程设计报告

1. 项目概述

本项目实现汽车销售管理系统，覆盖车辆目录、库存车辆、客户意向、销售订单、售后服务、销售报表和库存预警等业务。当前项目为 Web 应用：

后端：Spring Boot 3 + MyBatis。

前端：Vue 3 + Vite + Element Plus。

数据库：openGauss。

数据库脚本：位于 sql/ 目录。

文档：位于 doc/ 目录。

系统按照业务域拆分为销售前台、库存管理、报表中心三个主要模块。销售前台负责意向客户、销售订单和订单交付；库存管理负责车辆查询、车辆入库和库存预警；报表中心负责月度销售统计、销售顾问业绩排行和畅销车型排行。

2. 需求分析

2.1 业务对象

系统管理的核心业务对象包括品牌、车型、库存车辆、客户、员工、客户意向、销售订单、订单明细、服务工单、服务明细和车辆状态日志。

数据库中已将这些对象映射为独立关系表，并通过主键、外键、唯一约束和检查约束维护数据一致性。实体和关系模式的详细设计见《数据库设计报告》。

2.2 功能需求

当前代码实现的主要功能如下：

模块	已实现功能	对应代码位置
销售前台	创建意向客户、创建销售订单、查询我的订单、完成订单	SalesController、OrderServiceImpl、OrderMapper.xml、frontend/src/views/sales/SalesView.vue
库存管理	查询车辆库存、车辆入库、库存预警	InventoryController、InventoryServiceImpl、inventory_mapper.xml、

模块	已实现功能	对应代码位置
		frontend/src/views/inventory/InventoryView.vue
报表中心	销售业绩榜、畅销车型排行、月度销售统计	ReportController、ReportServiceImpl、ReportMapper.xml、frontend/src/views/report/ReportView.vue
数据库对象	表、视图、索引、触发器、存储过程、复杂查询	sql/01_create_schema.sql 至 sql/07_queries.sql

2.3 数据需求

数据库初始化脚本 sql/02_init_data.sql 提供了演示数据：

品牌不少于 3 个，实际包含 Mercedes-Benz、BMW、Audi、Porsche、Tesla。

车型不少于 10 款，实际包含 12 款。

库存车辆不少于 30 台，实际包含 32 台，覆盖在途、在库、锁定、已售状态。

员工不少于 10 人。

客户不少于 20 人。

销售订单不少于 20 条，实际包含 22 条。

服务工单不少于 10 条，实际包含 12 条。

2.4 约束需求

系统需要保证以下关键业务约束：

创建销售订单时，只允许销售状态为 IN_INVENTORY 的车辆。

订单创建后，车辆状态自动变为 LOCKED。

订单完成后，车辆状态自动变为 SOLD。

取消未完成订单时，车辆状态从 LOCKED 回滚为 IN_INVENTORY。

同一台车辆最多只能关联一张销售订单。

销售、库存、客户价值等统计数据应基于数据库当前数据实时计算。

上述约束由数据库检查约束、唯一约束、触发器、视图和存储过程共同实现。详情见《数据库设计报告》。

3. 总体设计

3.1 系统架构

系统采用前后端分离架构：



后端统一返回结构为：

```
{
  "code": 0,
  "message": "ok",
  "data": {}
}
```

前端通过 Axios 封装请求，按业务模块拆分 API 文件：

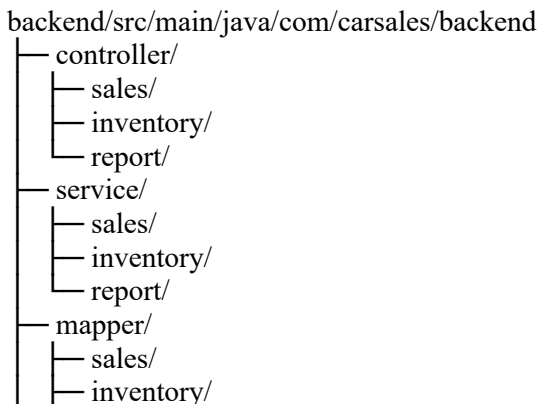
frontend/src/api/sales.js

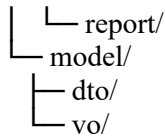
frontend/src/api/inventory.js

frontend/src/api/report.js

3.2 后端模块划分

后端按业务域划分 Controller、Service、Mapper、DTO、VO：





这种划分使销售、库存、报表三类功能边界清晰，减少模块间直接耦合。

3.3 前端模块划分

前端主要页面包括：

/sales：销售前台。

/inventory：库存管理。

/report：报表中心。

页面组件位于 `frontend/src/views/`，可复用组件位于 `frontend/src/components/`。表格、分页、筛选、弹窗等交互组件按业务域拆分。

4. 数据库设计概要

数据库设计包括 E-R 模型、3NF 关系模式、主外键、唯一约束、检查约束、视图、索引、触发器和存储过程。

本报告仅概述数据库设计要点：

基础表共 12 张，覆盖车辆、客户、员工、销售和售后业务。

车辆以 `vehicle_vin` 作为主键。

销售订单通过 `vehicle_vin` 关联具体实车，并通过唯一约束保证一车一单。

库存统计通过 `v_inventory_summary` 视图实时计算。

销售业绩通过 `v_sales_performance` 视图计算。

客户价值通过 `v_customer_value` 视图计算。

车辆状态联动通过 `trg_lock_car_on_order`、`trg_update_inventory_on_delivery`、`trg_prevent_invalid_order_cancel` 实现。

E-R 图、关系模式、字段类型、主外键、检查约束和索引设计已在数据库设计文档中详细说明。详情见《数据库设计报告》。

5. 功能实现

5.1 销售前台

销售前台对应后端 `/api/sales/*` 接口和前端 `SalesView.vue` 页面。

主要实现：

功能	实现方式
创建销售订单	后端调用存储过程 <code>sp_create_sales_order</code> ，数据库触发器负责锁定车辆
完成销售订单	后端更新订单状态为 <code>COMPLETED</code> ，数据库触发器负责将车辆更新为 <code>SOLD</code>
查询我的订单	MyBatis 动态 SQL 支持员工、客户、手机号、订单状态、VIN、时间范围筛选
创建意向客户	后端校验手机号、证件号、车型、员工，并写入客户与意向记录

关键代码：

`backend/src/main/java/com/carsales/backend/controller/sales/SalesController.java`

`backend/src/main/java/com/carsales/backend/service/sales/impl/OrderServiceImpl.java`

`backend/src/main/resources/mapper/sales/OrderMapper.xml`

`backend/src/main/resources/mapper/sales/CustomerIntentMapper.xml`

`frontend/src/views/sales/SalesView.vue`

5.2 库存管理

库存管理对应后端 `/api/inventory/*` 接口和前端 `InventoryView.vue` 页面。

主要实现：

功能	实现方式
车辆库存查询	支持 VIN、品牌、车型、状态、颜色、入库日期、生产日期筛选
车辆入库	仅允许 <code>IN_TRANSIT</code> 车辆更新为 <code>IN_INVENTORY</code> ，并写入车辆状态日志
库存预警	基于 <code>v_inventory_summary</code> 和 <code>model.safety_stock_threshold</code> 计算缺口

关键代码：

`backend/src/main/java/com/carsales/backend/controller/inventory/InventoryController.java`

backend/src/main/java/com/carsales/backend/service/inventory/impl/InventoryServiceImpl.java
backend/src/main/resources/mapper/inventory/inventory_mapper.xml
frontend/src/views/inventory/InventoryView.vue

5.3 报表中心

报表中心对应后端 `/api/report/*` 接口和前端 `ReportView.vue` 页面。

主要实现：

功能	实现方式
----	------

销售业绩榜	查询 <code>v_sales_performance</code> ，按销售额生成排名
畅销车型排行	对已完成订单按车型聚合，返回销量和销售额
月度销售统计	调用 <code>fn_get_monthly_report</code> ，后端负责分页

关键代码：

backend/src/main/java/com/carsales/backend/controller/report/ReportController.java
backend/src/main/java/com/carsales/backend/service/report/impl/ReportServiceImpl.java
backend/src/main/resources/mapper/report/ReportMapper.xml
frontend/src/views/report/ReportView.vue

6. 实现难点与解决方案

6.1 锁车逻辑重复导致职责不清

相关提交：0cc6c54 fix: delete repeated logic bewteen trigger and sp on lock car

问题：早期实现中，存储过程和触发器都包含车辆锁定逻辑，容易造成职责重复。如果两处逻辑同时维护，后续修改状态规则时可能出现不一致。

解决方案：

`sp_create_sales_order` 只负责订单和订单明细创建。

车辆从 `IN_INVENTORY` 到 `LOCKED` 的状态变更统一交给触发器 `trg_lock_car_on_order`。

触发器在 `BEFORE INSERT ON sales_order` 时执行，若车辆不是在库状态则抛出异常。

当前效果：销售订单创建和车辆锁定仍在同一数据库事务中完成，但职责边界更清晰。

6.2 销售业绩视图统计口径复杂

相关提交: f2a0505 fix: improve view v_sales_performance using GROUPING SET

问题: 销售业绩需要同时支持月度、季度、年度统计。如果分别写多个查询或多个视图, 会产生重复 SQL, 并增加维护成本。

解决方案:

在 v_sales_performance 中使用 GROUPING SETS。

一次聚合生成月度、季度、年度三种统计粒度。

使用 period_type、stat_year、stat_quarter、stat_month 标识统计周期。

当前效果: 报表接口只需基于同一个视图筛选周期类型, 即可支持月度和季度业绩排行。

6.3 滞销车辆查询口径错误

相关提交: 9c3b66d fix: Q4 bug

问题: 滞销车辆查询需要统计“当前仍在库且库存周期超过 90 天”的车辆。如果将已售出车辆或非在库车辆纳入, 会导致业务含义不准确。

解决方案:

Q4 查询限定 vehicle.current_status = 'IN_INVENTORY'。

要求 inventory_in_date IS NOT NULL。

使用 CURRENT_DATE - inventory_in_date > 90 计算库存周期。

当前效果: Q4 查询结果只反映仍占用库存的滞销车辆。

6.4 月报过程与后端调用方式不匹配

相关提交:

6c85d57 fix: create sp_get_monthly_report and add some comments

534a9c8 fix: fix jdbc failure

问题: 任务要求使用存储过程生成月度销售统计, 但 JDBC/MyBatis 对游标型存储过程调用不如普通 SELECT 直接。若后端直接处理游标, 接口实现复杂度较高。

解决方案:

保留存储过程 sp_get_monthly_report, 满足数据库对象要求。

增加表函数 `fn_get_monthly_report(year, month)`，返回可直接查询的表结构。

后端 `ReportMapper.xml` 使用 `SELECT * FROM fn_get_monthly_report(...)` 进行查询。

当前效果：数据库仍保留存储过程对象，同时后端可以用常规 `MyBatis` 查询方式获取月报数据。

6.5 报表分页需要由后端处理

相关提交：8af5fec `fix(report): backend handle page slice`

问题：月报结果需要分页。如果只在前端截取数据，数据量增大后会造成无效传输，也不利于接口统一。

解决方案：

`MonthlySalesReportQueryDto` 增加分页参数。

`ReportMapper.xml` 在查询月报时使用 `LIMIT` 和 `OFFSET`。

`ReportServiceImpl` 返回统一分页对象 `PageResult`。

前端 `ReportView.vue` 根据接口返回的 `total`、`pageNo`、`pageSize` 渲染分页。

当前效果：月报分页逻辑由后端负责，前端只负责展示和翻页请求。

6.6 库存入库需要防止状态并发变化

相关代码：`InventoryServiceImpl`、`inventory_mapper.xml`

问题：车辆入库只应作用于 `IN_TRANSIT` 车辆。如果车辆状态已被其他操作改变，继续入库会破坏状态流转规则。

解决方案：

后端先查询车辆当前状态和生产日期。

校验状态必须为 `IN_TRANSIT`。

更新 SQL 使用条件 `WHERE vehicle_vin = ? AND current_status = 'IN_TRANSIT'`。

如果更新行数为 0，返回冲突错误。

成功后写入 `vehicle_status_log`。

当前效果：车辆入库具有状态前置条件，并记录状态变更。

6.7 前端错误提示不足

相关提交: 27f23d9 fix: add error message

问题: 早期前端在请求失败时提示不充分, 尤其是库存查询失败时, 用户难以判断是后端异常、网络问题还是业务校验失败。

解决方案:

在 frontend/src/utils/request.js 中统一处理响应和错误。

在库存页面增加错误提示区域。

对网络错误和业务错误进行区分展示。

当前效果: 接口失败时前端能够显示更明确的错误信息。

6.8 数据库部署顺序需要固定

相关提交: 30ca702 feat: add deploy_all script

问题: 数据库对象之间存在依赖关系。例如视图依赖基础表, 触发器依赖表, 查询脚本依赖视图和存储过程。如果手动执行顺序错误, 会导致部署失败。

解决方案:

增加 sql/00_deploy_all.sql, 按顺序执行建表、初始化、视图、索引、触发器、存储过程、查询脚本。

增加 sql/00_reset_and_deploy.sql, 用于重置并重建 schema。

当前效果: 数据库初始化流程更稳定, 适合演示和重复测试。

7. 测试方案

Q1: 查询指定时间段内（先按月度）的销售统计

说明: 复用函数 sp_get_monthly_report（输入年份、月份）。

```
car_sales=# CALL fn_get_monthly_report(2026, 1);
```

stat_year	stat_month	staff_id	staff_name	order_count	sales_amount	gross_profit
2026	1	3	Wang Lei	4	1681300.00	328300.00
2026	1	5	Liu Yang	3	1480500.00	283500.00
2026	1	4	Chen Yu	3	1038500.00	190500.00

(3 rows)

Q2: 查询每位销售顾问的月度/季度业绩并排名

说明: 复用视图 v_sales_performance, 外层增加排名窗口函数。

```

car_sales=# SELECT
car_sales-#     period_type,
car_sales-#     stat_year,
car_sales-#     stat_quarter,
car_sales-#     stat_month,
car_sales-#     period_label,
car_sales-#     staff_id,
car_sales-#     staff_no,
car_sales-#     staff_name,
car_sales-#     order_count,
car_sales-#     sales_amount,
car_sales-#     gross_profit,
car_sales-#     DENSE_RANK() OVER (
car_sales-#         PARTITION BY period_type, stat_year, stat_quarter, stat_month
car_sales-#         ORDER BY sales_amount DESC
car_sales-#     ) AS sales_rank
car_sales-# FROM v_sales_performance
car_sales-# WHERE period_type IN ('MONTH', 'QUARTER')
car_sales-# ORDER BY period_type, stat_year, stat_quarter, stat_month, sales_rank, staff_id;
period_type | stat_year | stat_quarter | stat_month | period_label | staff_id | staff_no | staff_name | order_count | sales_amount | gross_profit | sales_rank
-----
MONTH       | 2026     | 1           | 1         | 2026-01     | 3       | S0003    | Wang Lei   | 4           | 1681300.00  | 328300.00   | 1
MONTH       | 2026     | 1           | 1         | 2026-01     | 5       | S0005    | Liu Yang   | 3           | 1480500.00  | 283500.00   | 2
MONTH       | 2026     | 1           | 1         | 2026-01     | 4       | S0004    | Chen Yu    | 3           | 1038500.00  | 190500.00   | 3
QUARTER    | 2026     | 1           |           | 2026-Q1     | 3       | S0003    | Wang Lei   | 4           | 1681300.00  | 328300.00   | 1
QUARTER    | 2026     | 1           |           | 2026-Q1     | 5       | S0005    | Liu Yang   | 3           | 1480500.00  | 283500.00   | 2
QUARTER    | 2026     | 1           |           | 2026-Q1     | 4       | S0004    | Chen Yu    | 3           | 1038500.00  | 190500.00   | 3
(6 rows)

```

Q3: 查询最畅销车型 Top 5 及销量

说明：原生 SQL，按 COMPLETED 订单统计车型销量。

```

car_sales=# SELECT
car_sales-#     b.brand_name,
car_sales-#     m.model_name,
car_sales-#     m.model_year,
car_sales-#     m.trim_name,
car_sales-#     COUNT(*)::INT AS sales_volume
car_sales-# FROM sales_order so
car_sales-# JOIN vehicle v ON v.vehicle_vin = so.vehicle_vin
car_sales-# JOIN model m ON m.model_id = v.model_id
car_sales-# JOIN brand b ON b.brand_id = m.brand_id
car_sales-# WHERE so.order_status = 'COMPLETED'
car_sales-# GROUP BY b.brand_name, m.model_name, m.model_year, m.trim_name
car_sales-# ORDER BY sales_volume DESC, b.brand_name, m.model_name
car_sales-# LIMIT 5;
brand_name | model_name | model_year | trim_name | sales_volume
-----
Mercedes-Benz | C-Class | 2026 | C 260 L Sport | 2
Audi | A4L | 2026 | 40 TFSI Luxury | 1
Audi | A6L | 2026 | 45 TFSI Quattro | 1
Audi | Q5L | 2026 | 45 TFSI Luxury | 1
BMW | 3 Series | 2026 | 325Li M Sport | 1
(5 rows)

```

Q4: 查询库存周期超过 90 天的滞销车辆清单

说明：按当前库存统计，仅包含“仍在库(IN_INVENTORY)”且入库超 90 天的车辆。

```

car_sales=# SELECT
car_sales-#     v.vehicle_vin,
car_sales-#     b.brand_name,
car_sales-#     m.model_name,
car_sales-#     m.model_year,
car_sales-#     m.trim_name,
car_sales-#     v.inventory_in_date,
car_sales-#     (CURRENT_DATE - v.inventory_in_date) AS inventory_days
car_sales-# FROM vehicle v
car_sales-# JOIN model m ON m.model_id = v.model_id
car_sales-# JOIN brand b ON b.brand_id = m.brand_id
car_sales-# WHERE v.current_status = 'IN_INVENTORY'
car_sales-# AND v.inventory_in_date IS NOT NULL
car_sales-# AND (CURRENT_DATE - v.inventory_in_date) > 90
car_sales-# ORDER BY inventory_days DESC, v.inventory_in_date ASC, v.vehicle_vin;

```

vehicle_vin	brand_name	model_name	model_year	trim_name	inventory_in_date	inventory_days
VIN00000000000021	BMW	3 Series	2026	325Li M Sport	2026-01-24 00:00:00	125 days
VIN00000000000022	Audi	A4L	2026	40 TFSI Luxury	2026-01-25 00:00:00	124 days
VIN00000000000023	Audi	Q5L	2026	45 TFSI Luxury	2026-01-26 00:00:00	123 days
VIN00000000000024	Porsche	Macan	2026	Base	2026-01-27 00:00:00	122 days
VIN00000000000025	Tesla	Model Y	2026	Long Range AWD	2026-01-28 00:00:00	121 days
VIN00000000000026	BMW	5 Series	2026	530Li xDrive	2026-01-29 00:00:00	120 days
VIN00000000000027	Mercedes-Benz	GLC SUV	2026	GLC 260 L 4MATIC	2026-01-30 00:00:00	119 days
VIN00000000000028	BMW	X3	2026	xDrive30i M Sport	2026-01-31 00:00:00	118 days

(8 rows)

Q5: 根据客户历史消费总额进行客户分类

说明：复用视图 v_customer_value。

```

car_sales=# SELECT
car_sales-#     customer_id,
car_sales-#     customer_name,
car_sales-#     phone,
car_sales-#     purchase_order_count,
car_sales-#     purchase_total_amount,
car_sales-#     service_order_count,
car_sales-#     service_total_fee,
car_sales-#     customer_level
car_sales-# FROM v_customer_value
car_sales-# ORDER BY purchase_total_amount DESC, customer_id;

```

customer_id	customer_name	phone	purchase_order_count	purchase_total_amount	service_order_count	service_total_fee	customer_level
1006	Sun Mei	13800000006	1	506900.00	0	0.00	GOLD
1003	Wang Bin	13800000003	1	497800.00	1	980.00	GOLD
1009	Xu Lei	13800000009	1	475800.00	1	1120.00	GOLD
1007	Guo Peng	13800000007	1	447900.00	0	0.00	GOLD
1004	Zhao Xin	13800000004	1	438800.00	0	0.00	GOLD
1010	Tang Yue	13800000010	1	427800.00	0	0.00	GOLD
1001	Chen Ming	13800000001	1	366800.00	1	1280.00	GOLD
1002	Li Na	13800000002	1	359800.00	1	3560.00	GOLD
1005	Liu Qiang	13800000005	1	344900.00	1	760.00	GOLD
1008	He Fang	13800000008	1	333800.00	1	640.00	GOLD
1011	Deng Bo	13800000011	0	0.00	0	0.00	REGULAR
1012	Yuan Jing	13800000012	0	0.00	1	980.00	REGULAR
1013	Cao Rui	13800000013	0	0.00	0	0.00	REGULAR
1014	Qin Lan	13800000014	0	0.00	0	0.00	REGULAR
1015	Hu Tao	13800000015	0	0.00	0	0.00	REGULAR
1016	Song Yi	13800000016	0	0.00	0	0.00	REGULAR
1017	Pan Hao	13800000017	0	0.00	0	0.00	REGULAR
1018	Zheng Min	13800000018	0	0.00	0	0.00	REGULAR
1019	Feng Kai	13800000019	0	0.00	0	0.00	REGULAR
1020	Jiang Xin	13800000020	0	0.00	0	0.00	REGULAR

(20 rows)

Q6: 查询特定客户的完整购车及服务历史

说明：复用存储过程 sp_get_customer_full_history。

参数示例：customer_id = 1001

```
car_sales=# BEGIN;
BEGIN
car_sales=# CALL sp_get_customer_full_history(1001, 'cur_customer_history');
p_result
-----
cur_customer_history
(1 row)

car_sales=# FETCH ALL FROM cur_customer_history;
customer_id | event_time | event_type | ref_no | status | amount | vehicle_vin | staff_name
-----
1001 | 2026-01-20 10:00:00 | SALES_ORDER | 5001 | COMPLETED | 366800.00 | VIN0000000000000001 | Wang Lei
1001 | 2026-02-18 10:15:00 | SALES_ORDER | 5021 | CANCELLED | 342900.00 | VIN0000000000000021 | Liu Yang
1001 | 2026-03-01 09:00:00 | SERVICE_ORDER | 9001 | COMPLETED | 1280.00 | VIN0000000000000001 | Ma Chao
(3 rows)

car_sales=# COMMIT;
COMMIT
```

Q7: 生成库存预警报表（库存低于安全库存阈值）

说明：复用视图 v_inventory_summary + model.safety_stock_threshold

```
car_sales=# SELECT
car_sales-# inv.model_id,
car_sales-# inv.brand_name,
car_sales-# inv.model_name,
car_sales-# inv.model_year,
car_sales-# inv.trim_name,
car_sales-# m.safety_stock_threshold,
car_sales-# inv.in_inventory_count,
car_sales-# (m.safety_stock_threshold - inv.in_inventory_count) AS shortage_count
car_sales-# FROM v_inventory_summary inv
car_sales-# JOIN model m ON m.model_id = inv.model_id
car_sales-# WHERE inv.in_inventory_count < m.safety_stock_threshold
car_sales-# ORDER BY shortage_count DESC, inv.brand_name, inv.model_name;
model_id | brand_name | model_name | model_year | trim_name | safety_stock_threshold | in_inventory_count | shortage_count
-----
501 | Tesla | Model Y | 2026 | Long Range AWD | 4 | 1 | 3
303 | Audi | Q5L | 2026 | 45 TFSI Luxury | 3 | 1 | 2
101 | Mercedes-Benz | C-Class | 2026 | C 260 L Sport | 2 | 0 | 2
102 | Mercedes-Benz | E-Class | 2026 | E 300 L Luxury | 2 | 0 | 2
103 | Mercedes-Benz | GLC SUV | 2026 | GLC 260 L 4MATIC | 3 | 1 | 2
301 | Audi | A4L | 2026 | 40 TFSI Luxury | 2 | 1 | 1
302 | Audi | A6L | 2026 | 45 TFSI Quattro | 2 | 1 | 1
201 | BMW | 3 Series | 2026 | 325Li M Sport | 2 | 1 | 1
202 | BMW | 5 Series | 2026 | 530Li xDrive | 2 | 1 | 1
203 | BMW | X3 | 2026 | xDrive30i M Sport | 2 | 1 | 1
401 | Porsche | Cayenne | 2026 | Base | 1 | 0 | 1
(11 rows)
```

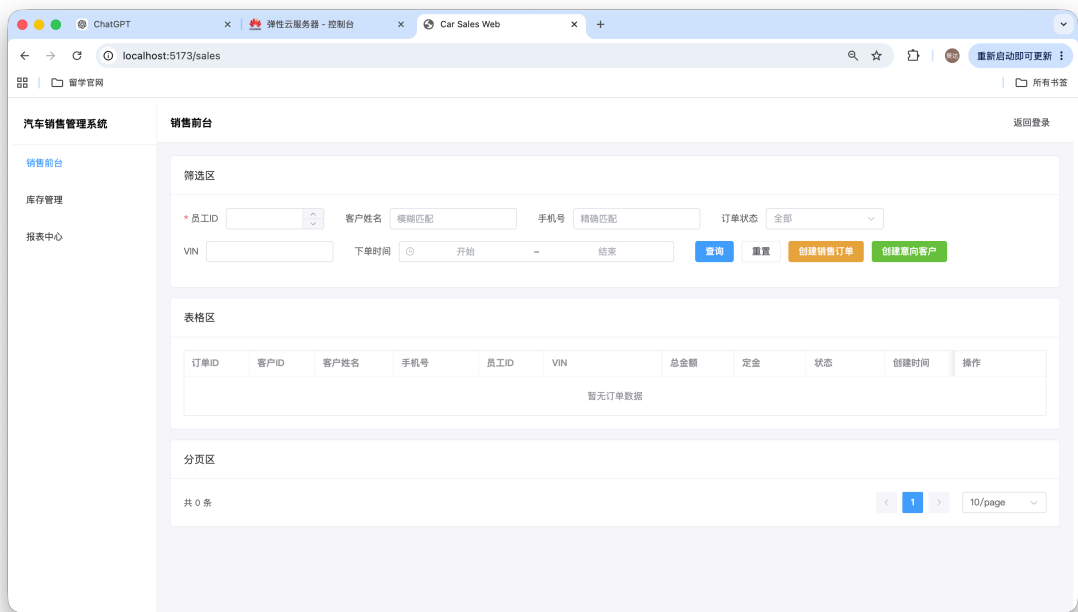
Q8: 自定义复杂查询（业务价值）

题目：销售顾问订单取消率与潜在损失分析（识别高风险顾问）

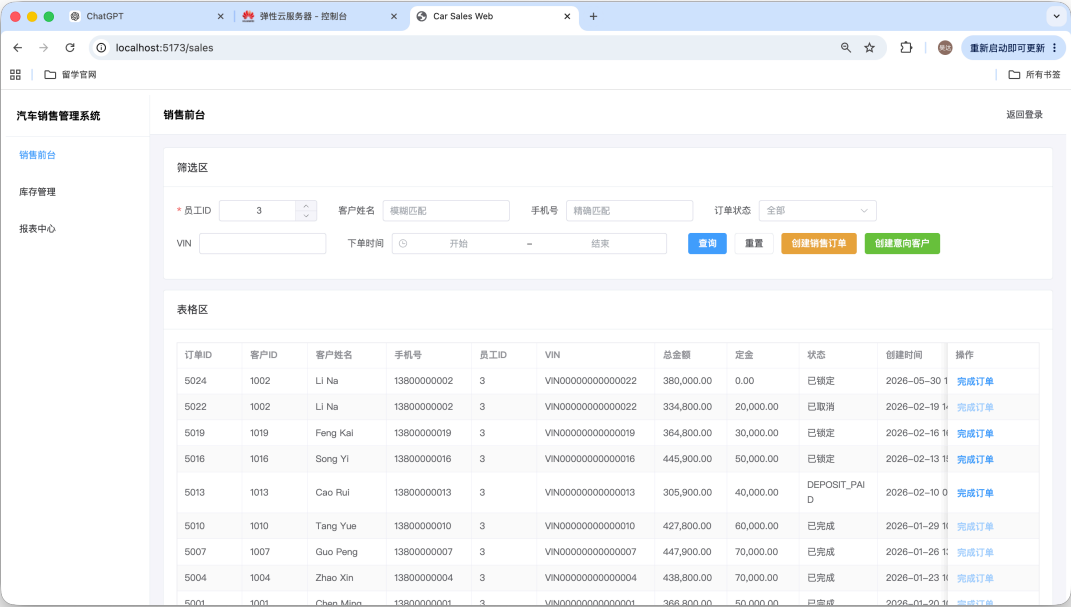
```
car_sales=# SELECT
car_sales-#     st.staff_id,
car_sales-#     st.staff_name,
car_sales-#     COUNT(*)::INT AS total_orders,
car_sales-#     SUM(CASE WHEN so.order_status = 'CANCELLED' THEN 1 ELSE 0 END)::INT AS cancelled_orders,
car_sales-#     ROUND(
car_sales-#         SUM(CASE WHEN so.order_status = 'CANCELLED' THEN 1 ELSE 0 END)::NUMERIC
car_sales-#         / NULLIF(COUNT(*), 0),
car_sales-#         4
car_sales-#     ) AS cancel_rate,
car_sales-#     COALESCE(
car_sales-#         SUM(CASE WHEN so.order_status = 'CANCELLED' THEN so.total_amount ELSE 0 END),
car_sales-#         0
car_sales-#     )::NUMERIC(14, 2) AS cancelled_amount
car_sales-# FROM sales_order so
car_sales-# JOIN staff st ON st.staff_id = so.staff_id
car_sales-# GROUP BY st.staff_id, st.staff_name
car_sales-# HAVING COUNT(*) ≥ 1
car_sales-# ORDER BY cancel_rate DESC, cancelled_amount DESC, total_orders DESC;
 staff_id | staff_name | total_orders | cancelled_orders | cancel_rate | cancelled_amount
-----+-----+-----+-----+-----+-----
      5 | Liu Yang   |           7 |               1 |       .1429 |      342900.00
      3 | Wang Lei   |           8 |               1 |       .1250 |      334800.00
      4 | Chen Yu    |           7 |               0 |       0.0000 |           0.00
(3 rows)
```

销售前台测试：

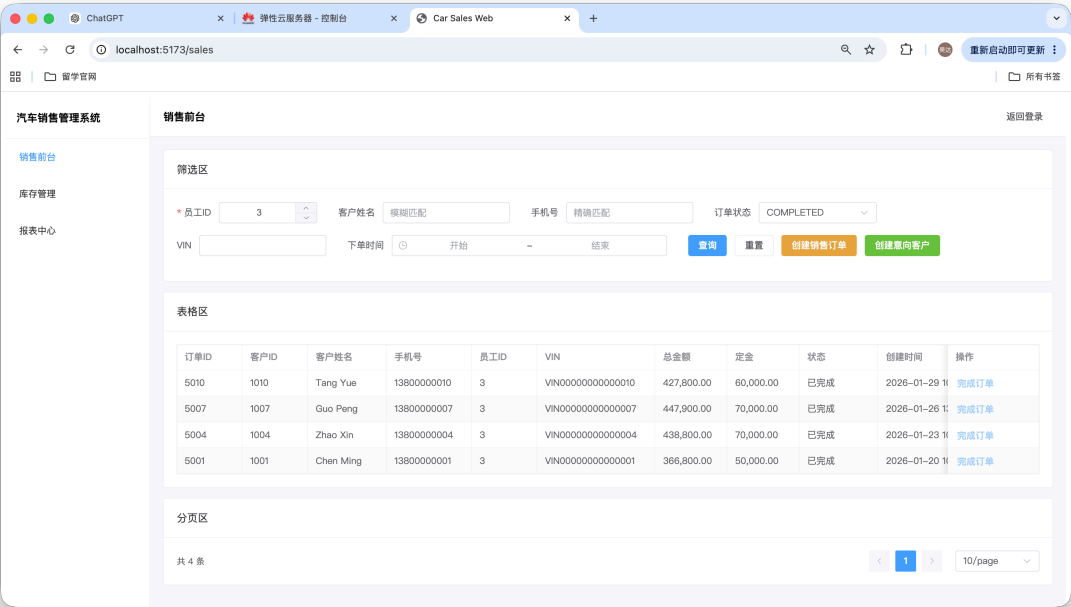
主页面：



输入员工 ID 查询我的订单：

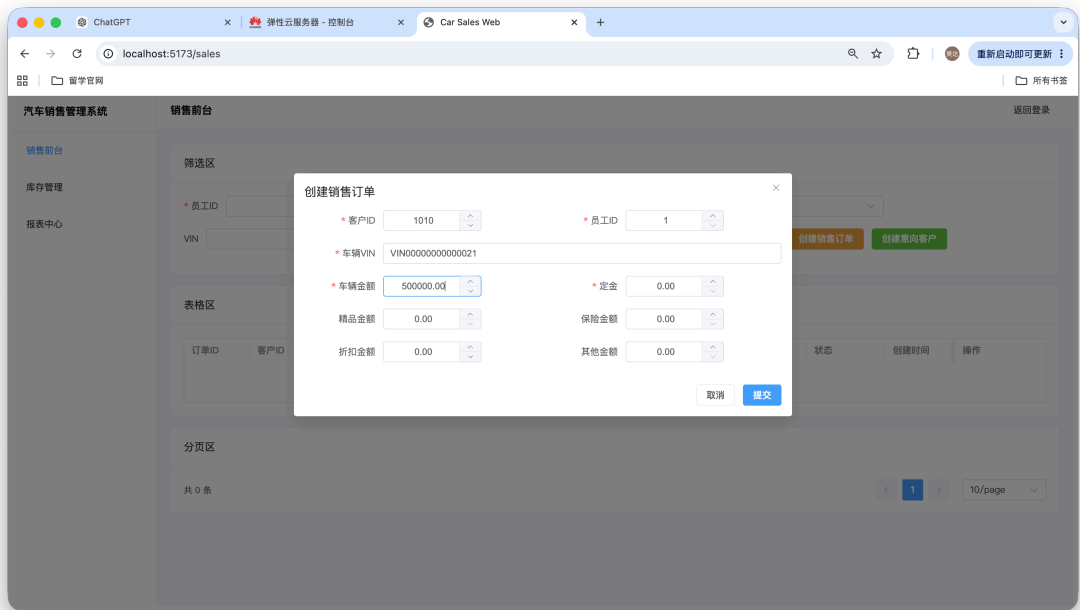


查询“已完成”订单：

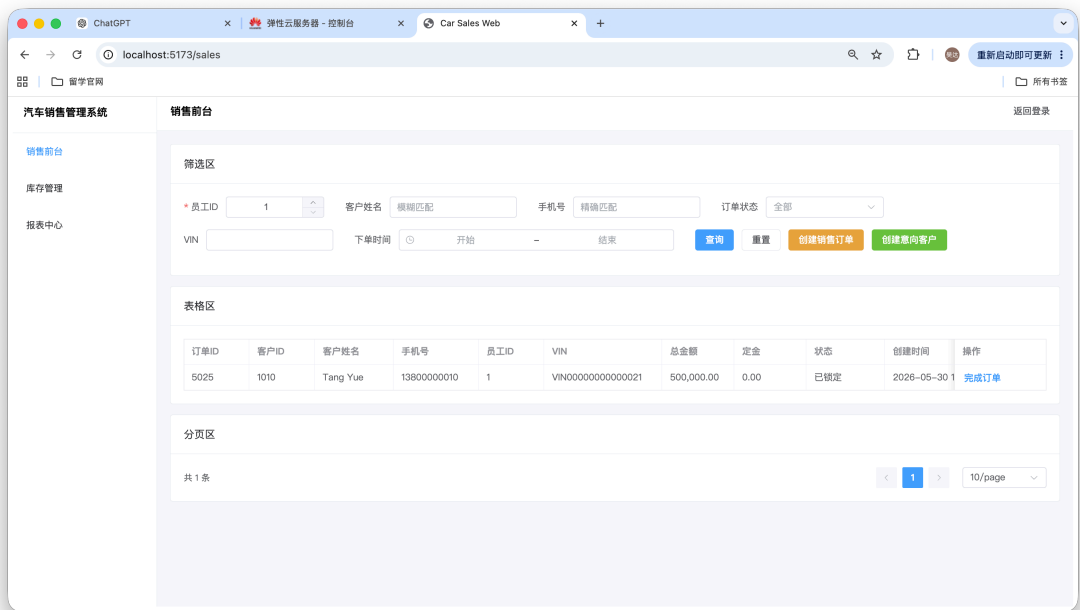


创建销售订单：

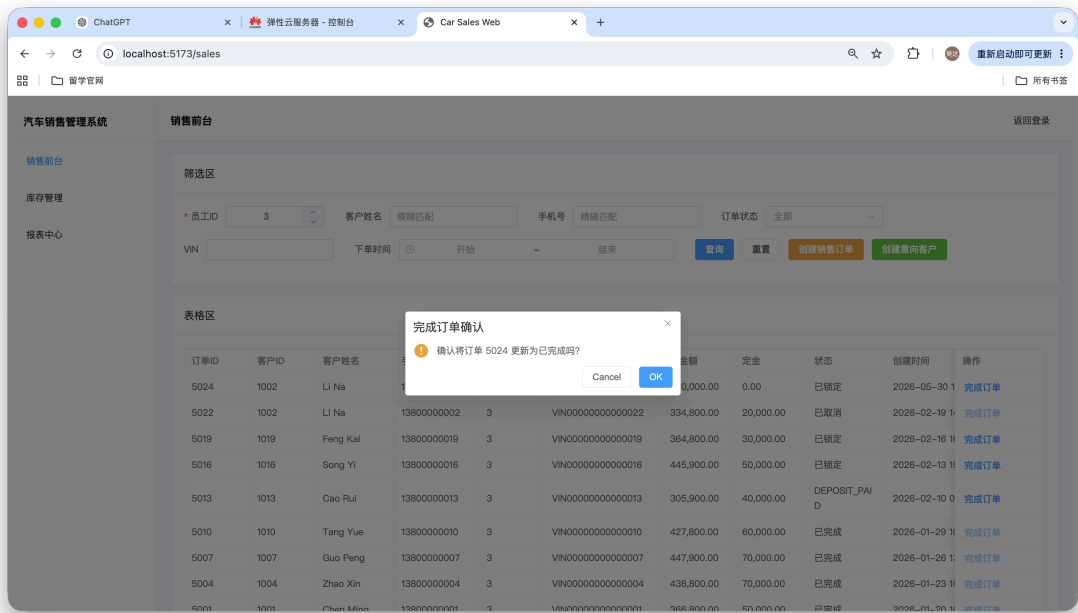
输入基本信息：



提交订单后再次查询（下单后自动锁车）：

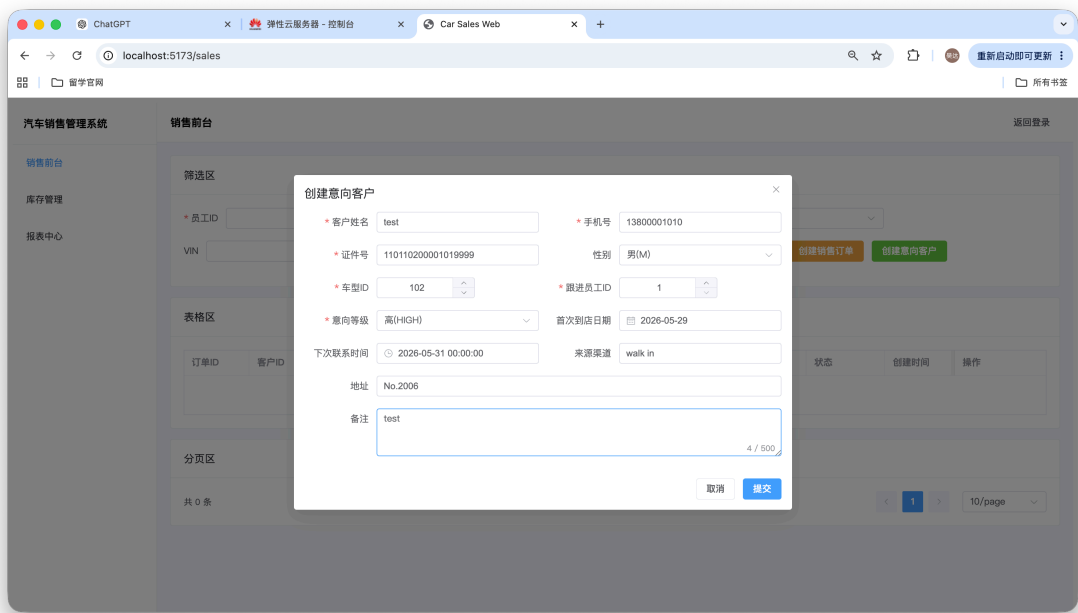


完成订单（完成后将车辆状态改为“已售出”）：



创建意向用户：

填写基本信息：

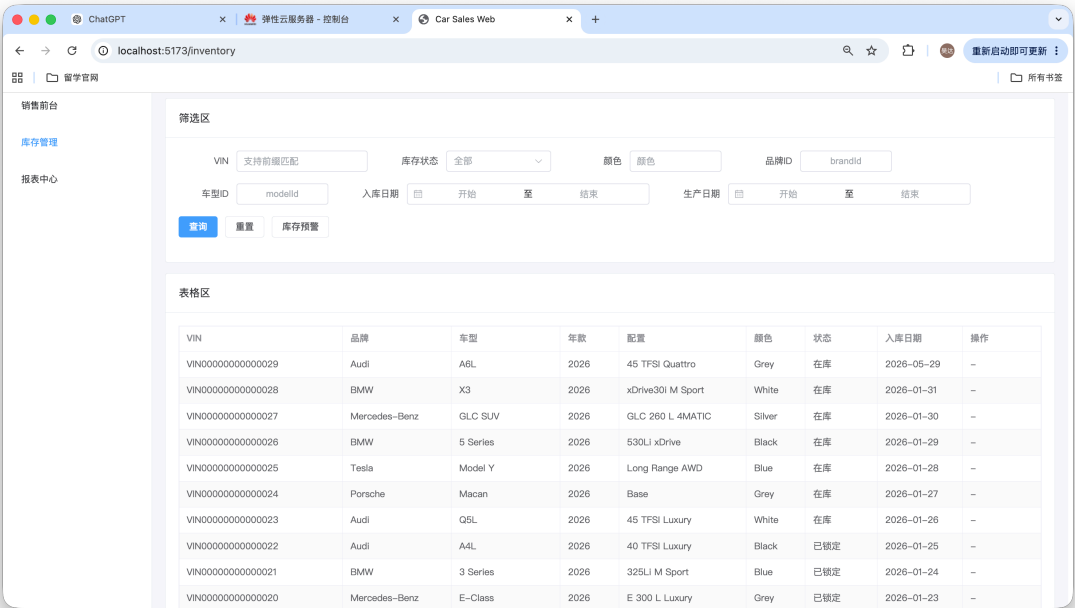


创建后，通过数据库查询验证：

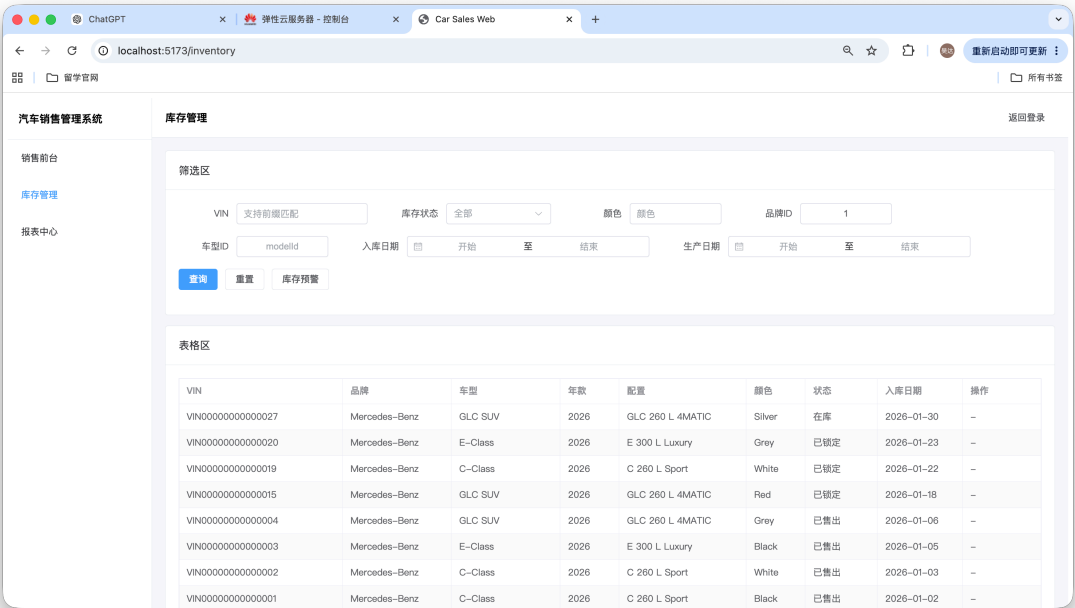

```
car_sales=# select * from customer_intent where model_id = 102 and staff_id = 1;
intent_id | customer_id | model_id | intent_level | remark | staff_id | next_contact_time | created_at
-----+-----+-----+-----+-----+-----+-----+-----
21 | 1021 | 102 | HIGH | test | 1 | 2026-05-31 00:00:00 | 2026-05-30 10:37:04.309847
(1 row)
```

库存管理功能验证：

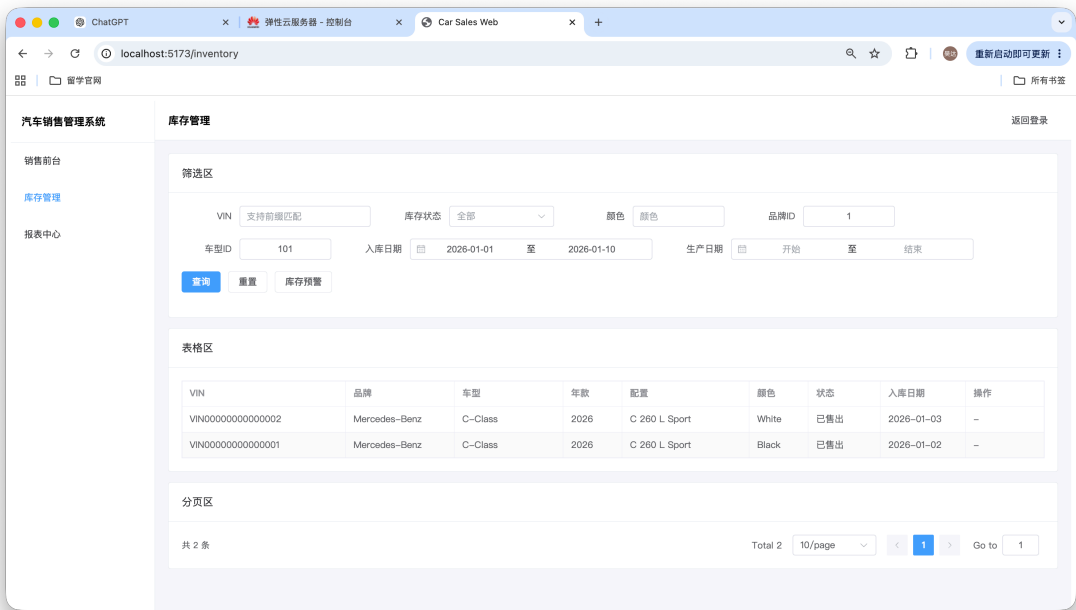
可以看到之前订单涉及到的 VIN000000000000021 已经售出



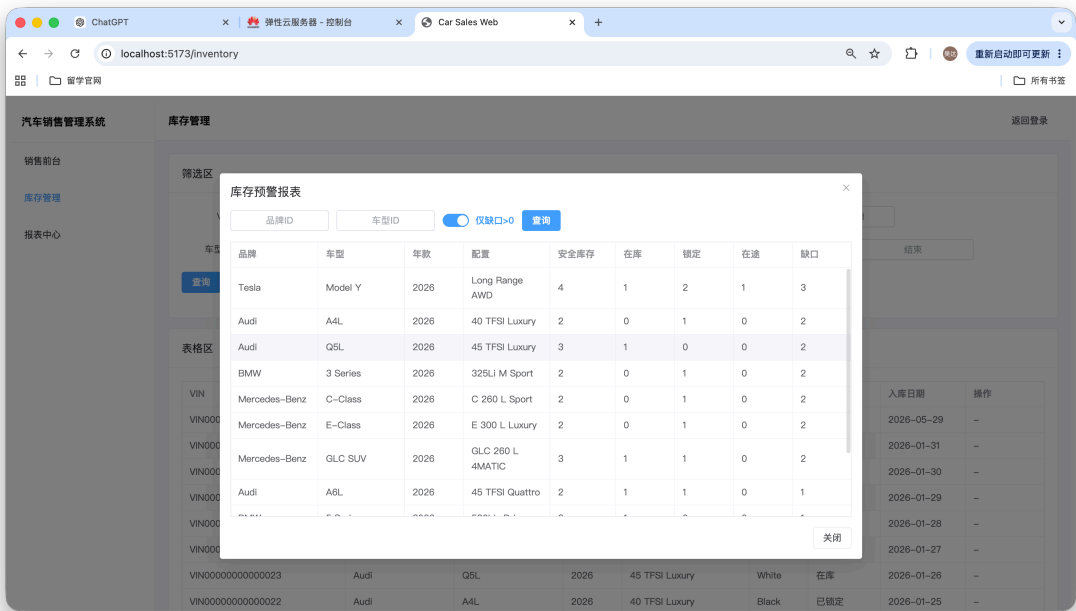
按照品牌 id 筛选（id = 1）



多条件筛选：

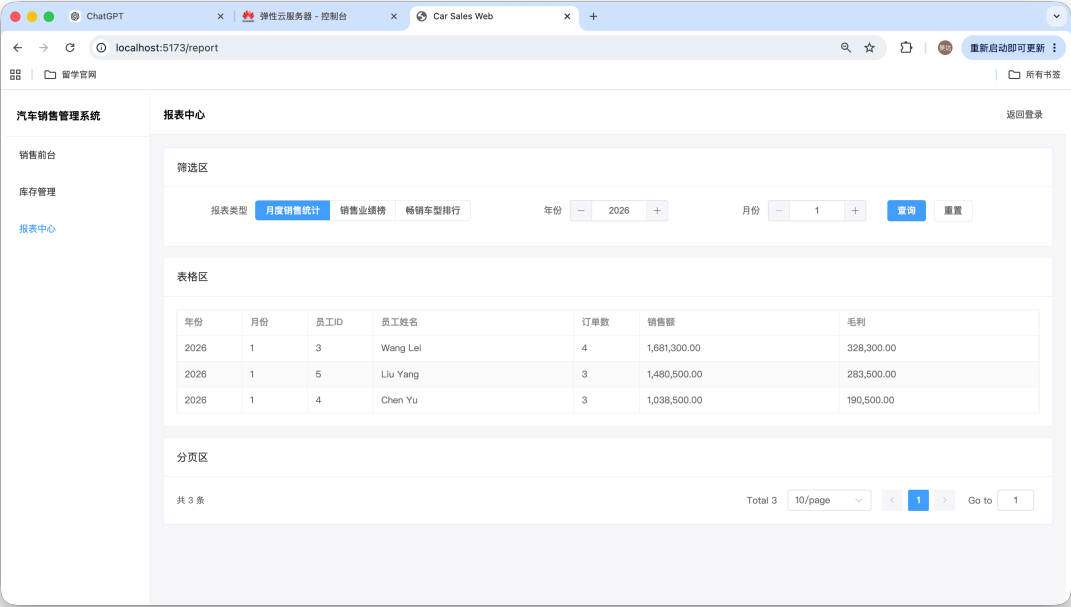


查询库存预警报表：

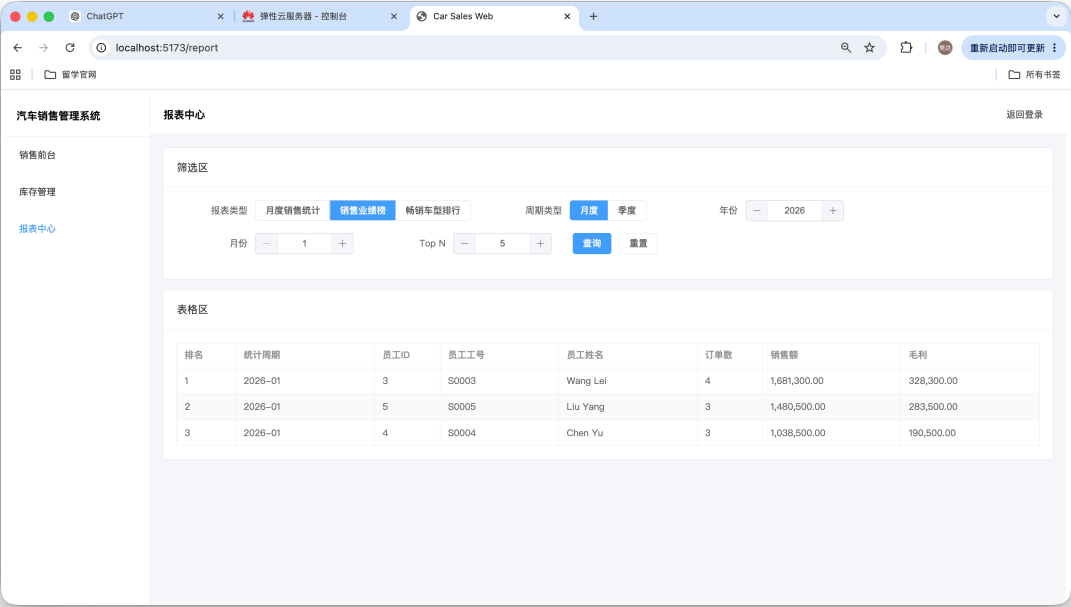


报表中心功能验证：

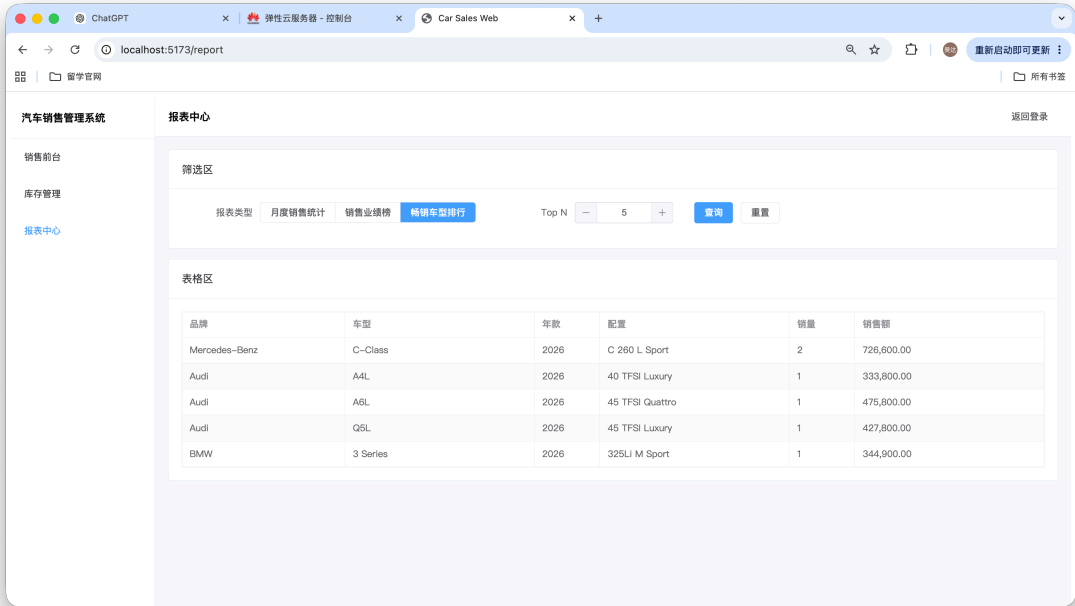
月度销售统计：



查询销售业绩排行（按月度或者季度查询）：



畅销车型排行查询：



数据库查询状态变化日志：

```
car_sales=# select * from vehicle_status_log;
log_id | vehicle_vin | from_status | to_status | changed_at | staff_id | reason
-----+-----+-----+-----+-----+-----+-----
11001 | VIN000000000000001 | IN_INVENTORY | LOCKED | 2026-01-20 10:05:00 | 3 | Order created
11002 | VIN000000000000001 | LOCKED | SOLD | 2026-01-25 15:00:00 | 3 | Vehicle delivered
11003 | VIN000000000000011 | IN_INVENTORY | LOCKED | 2026-02-08 11:02:00 | 4 | Deposit paid
11004 | VIN000000000000029 |  | IN_TRANSIT | 2026-01-20 09:00:00 | 6 | Purchased from factory
11005 | VIN000000000000023 | IN_TRANSIT | IN_INVENTORY | 2026-01-26 10:00:00 | 7 | Arrived at dealership
11006 | VIN000000000000029 | IN_TRANSIT | IN_INVENTORY | 2026-05-29 22:31:48.981307 | 1 | 车辆入库测试
11007 | VIN000000000000022 | IN_INVENTORY | LOCKED | 2026-05-30 10:25:33.672621 | 3 | Order created
11008 | VIN000000000000021 | IN_INVENTORY | LOCKED | 2026-05-30 10:34:02.936104 | 1 | Order created
11009 | VIN000000000000022 | LOCKED | SOLD | 2026-05-30 10:52:18.371536 | 3 | Order delivered
11010 | VIN000000000000022 | LOCKED | SOLD | 2026-05-30 11:03:10.348679 | 3 | Order delivered
(10 rows)
```

8. 结论与总结

本项目已实现汽车销售管理系统的核心数据库设计和 Web 应用功能。数据库层通过规范化表结构、约束、视图、触发器和存储过程支撑销售、库存、售后与报表业务；后端通过 Spring Boot 和 MyBatis 对数据库对象进行接口封装；前端通过 Vue 3 和 Element Plus 提供可操作页面。

从实现过程看，项目中较有价值的问题主要集中在数据库业务规则边界、统计查询口径、存储过程与后端调用兼容、分页处理和错误提示等方面。当前实现已经针对这些问题进行了修正。